

Dataset Summary

Data Size: *Approximately 1,000+ rows × 20 columns*

Includes :

Applicant Information

- Age
- Employment status
- Annual income
- Credit score

Loan Details

- Loan amount
- Loan purpose
- Interest rate

Financial Indicators

- Existing debts
- Debt-to-income factors
- Credit history length

Outcome Variable

- Loan status (Approved / Rejected) (numerical format 1/0)

Python-Based Exploratory Data Analysis

Workflow Overview – Data Preparation and Cleaning in Python

The project begins with data preparation and exploration in Python to ensure the dataset is clean, structured, and ready for deeper analysis.

• Data Loading

The dataset is imported using **pandas** for cleaning, and exploratory analysis.

```
df=pd.read_csv('Loan_approval_data_2025.csv')
df.head()
```

	customer_id	age	occupation_status	years_employed	annual_income	credit_score	credit_history_years	savings_assets	current_debt	defaults_on_file
0	CUST100000	40	Employed	17.2	25579	692	5.3	895	10820	0
1	CUST100001	33	Employed	7.3	43087	627	3.5	169	16550	0
2	CUST100002	42	Student	1.1	20840	689	8.4	17	7852	0
3	CUST100003	53	Student	0.5	29147	692	9.8	1480	11603	0
4	CUST100004	32	Employed	12.5	63657	630	7.2	209	12424	0

• Initial Exploration

Basic structural and statistical checks are performed:

- `df.info()` to check data types, null values, and column structure

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 20 columns):
#   Column                                Non-Null Count  Dtype  
---  -
0   customer_id                          50000 non-null  object  
1   age                                  50000 non-null  int64   
2   occupation_status                    50000 non-null  object  
3   years_employed                      50000 non-null  float64  
4   annual_income                       50000 non-null  int64   
5   credit_score                        50000 non-null  int64   
6   credit_history_years                 50000 non-null  float64  
7   savings_assets                      50000 non-null  int64   
8   current_debt                       50000 non-null  int64   
9   defaults_on_file                    50000 non-null  int64   
10  delinquencies_last_2yrs             50000 non-null  int64   
11  derogatory_marks                    50000 non-null  int64   
12  product_type                        50000 non-null  object  
13  loan_intent                         50000 non-null  object  
14  loan_amount                        50000 non-null  int64   
15  interest_rate                      50000 non-null  float64  
16  debt_to_income_ratio                50000 non-null  float64  
17  loan_to_income_ratio                50000 non-null  float64  
18  payment_to_income_ratio             50000 non-null  float64  
19  loan_status                        50000 non-null  int64   
dtypes: float64(6), int64(10), object(4)
```

- Checked for null values

```
# checking for null values
```

```
df.isnull().sum()
```

```
customer_id      0
age              0
occupation_status 0
years_employed   0
annual_income     0
credit_score      0
credit_history_years 0
savings_assets    0
current_debt      0
defaults_on_file  0
delinquencies_last_2yrs 0
derogatory_marks  0
product_type      0
loan_intent       0
loan_amount       0
interest_rate     0
debt_to_income_ratio 0
loan_to_income_ratio 0
payment_to_income_ratio 0
loan_status       0
dtype: int64
```

- Added some new columns for easy analysis and visualization

```
[118]: # Creating some New columns 1. age group
```

```
df['age_group']=pd.qcut(df['age'],q=4,labels=['Young','Young Adults','Middle-aged','Older adults'])
```

```
[120]: df[['age','age_group']].head()
```

```
[120]:
```

	age	age_group
0	40	Middle-aged
1	33	Young Adults
2	42	Middle-aged
3	53	Older adults
4	32	Young Adults

```
# 2.Credit score segmentation
l=[]
for i in df['credit_score']:
    if i <580:
        l.append('Low')
    elif i>=580 and i<670:
        l.append('Fair')
    elif i>=670 and i<740:
        l.append('Good')
    elif i>=740 :
        l.append('Excellent')

df['credit_tier']=l

df[['credit_score','credit_tier']].head()
```

	credit_score	credit_tier
0	692	Good
1	627	Fair
2	689	Good
3	692	Good
4	630	Fair

```
# 3. income bucket column

df['income_bucket']=pd.qcut(df['annual_income'],q=3,labels=['Low income','Middle income','High income'])

df[['annual_income','income_bucket']]
```

	annual_income	income_bucket
0	25579	Low income
1	43087	Middle income
2	20840	Low income
3	29147	Low income
4	63657	High income
...	...	...

- Re-arranged columns for better relatability

```
[124]: df=df[['customer_id', 'age', 'age_group','occupation_status', 'years_employed',
'annual_income', 'credit_score', 'credit_history_years', 'credit_tier',
'savings_assets', 'current_debt', 'defaults_on_file',
'delinquencies_last_2yrs', 'derogatory_marks', 'product_type',
'loan_intent', 'loan_amount', 'interest_rate', 'debt_to_income_ratio',
'loan_to_income_ratio', 'payment_to_income_ratio', 'loan_status']]

df.head()
```

```
[124]:
```

	customer_id	age	age_group	occupation_status	years_employed	annual_income	credit_score	credit_history_years	credit_tier	savings_assets
0	CUST100000	40	Middle-aged	Employed	17.2	25579	692	5.3	Good	895
1	CUST100001	33	Young Adults	Employed	7.3	43087	627	3.5	Fair	169
2	CUST100002	42	Middle-aged	Student	1.1	20840	689	8.4	Good	17
3	CUST100003	53	Older adults	Student	0.5	29147	692	9.8	Good	1480
4	CUST100004	32	Young Adults	Employed	12.5	63657	630	7.2	Fair	209

Activa

Connecting to MySQL for Deeper Analysis

Connecting to MySql for deeper analysis

```
from sqlalchemy import create_engine

# MySQL connection
username = "root"
password = "admin"
host = "localhost"
port = "3306"
database = "Project"

engine = create_engine(f"mysql+pymysql://{username}:{password}@{host}:{port}/{database}")

# Write DataFrame to MySQL
table_name = "loan_approval"
df.to_sql(table_name, engine, if_exists="replace", index=False)
```

- customer segments (age_group + occupation_status) having the highest loan approval rate

	age_group	occupation_status	total_customers	approved_loans	approval_rate
▶	Older adults	Employed	8023	6004	74.83
	Older adults	Self-Employed	2317	1722	74.32
	Older adults	Student	1064	736	69.17
	Middle-aged	Self-Employed	2440	1566	64.18
	Middle-aged	Employed	8279	5254	63.46
	Middle-aged	Student	1196	742	62.04
	Young Adults	Self-Employed	2879	1612	55.99
	Young Adults	Student	1377	737	53.52
	Young Adults	Employed	9749	5017	51.46
	Young	Student	1213	506	41.71
	Young	Self-Employed	2543	893	35.12
	Young	Employed	8920	2734	30.65

- product types generating the most approved loan volume

	product_type	Total_customers	Avg_loan_amount	loans_approved	Total_approved_volume
▶	Credit Card	22455	29263.05	13770	400115500
	Personal Loan	17523	44278.18	8417	373372600
	Line of Credit	10022	21862.44	5336	119043200

- Which credit score range brings the highest revenue potential?

	credit_tier	Total_customers	loans_approved	loan_approved_PCT
▶	Fair	24657	12368	50.1602
	Good	13779	10967	79.5921
	Excellent	3526	3116	88.3721
	Low	8038	1072	13.3367

- loan intent (purpose) having the lowest default risk

	loan_intent	Total_customers	sum_of_defaults	default_PCT
▶	Home Improvement	7453	382	5.1
	Business	7469	387	5.2
	Debt Consolidation	4917	263	5.3
	Medical	7598	406	5.3
	Personal	12429	675	5.4
	Education	10134	561	5.5

- Do high-income customers still default?

	income_bucket	Avg_income	Total_customers	defaults	Default_rate
▶	High income	85573.99	16666	726	4.3562
	Middle income	42076.24	16667	935	5.6099
	Low income	22540.57	16667	1013	6.0779

- customer groups paying the highest interest rate on approved loans

	age_group	occupation_status	avg_interest_rate
▶	Young	Student	16.6
	Young	Employed	16.5
	Young Adults	Student	16.1
	Young	Self-Employed	16.1
	Young Adults	Employed	15.8
	Middle-aged	Student	15.6
	Young Adults	Self-Employed	15.6
	Middle-aged	Employed	15.2
	Middle-aged	Self-Employed	15.2
	Older adults	Student	14.5
	Older adults	Employed	14.4
	Older adults	Self-Employed	14.4

- customers having the capacity to borrow more? (low debt-to-income + approved)

	customer_id	annual_income	debt_to_income_ratio	loan_amount
▶	CUST134058	76696	0.002	18500
	CUST107670	43661	0.003	27000
	CUST107421	53624	0.003	36000
	CUST137475	22731	0.003	2300
	CUST132292	43326	0.004	29200
	CUST108937	36952	0.004	1200
	CUST149097	39783	0.004	54900
	CUST139743	68814	0.005	38000
	CUST103842	34370	0.005	22600
	CUST102210	18616	0.005	8600

- Do credit score and interest rate align? (pricing leakage detection)

	credit_score	Avg_interest_rate
▶ 348	21.23	
375	16.01	
396	21.76	
401	21.27	
402	22.29	
408	16.96	
410	22.15	
414	14.33	
421	16.87	
422	15.14	
423	22.28	
424	16.56	
425	22.06	
428	17.04	
430	14.19	

- Which age groups default the most?

age_group	Total_customers	defaulters	Default_rate
Young	12676	938	7.40
Young Adults	14005	755	5.39
Middle-aged	11915	579	4.86
Older adults	11404	402	3.53

- occupation group bringing the most loan revenue

	occupation_status	Total_customers	loan_approval	Avg_payment_to_income_ratio
▶	Employed	34971	19009	0.25
	Self-Employed	10179	5793	0.23
	Student	4850	2721	0.16

- customers likely underpriced? (High risk but low interest rate)

	customer_id	credit_score	credit_tier	delinquencies_last_2yrs	defaults_on_file	interest_rate
▶	CUST131271	463	Low	4	1	13.01
	CUST122007	572	Low	3	1	13.03
	CUST126824	533	Low	3	1	13.06
	CUST116215	596	Fair	3	1	13.09
	CUST115295	488	Low	4	1	13.12
	CUST112123	496	Low	6	1	13.13
	CUST125353	564	Low	3	1	13.13
	CUST128260	585	Fair	4	1	13.15
	CUST136301	571	Low	3	1	13.26
	CUST141793	556	Low	3	1	13.26
	CUST132648	545	Low	5	1	13.33
	CUST108365	550	Low	3	1	13.36
	CUST138430	562	Low	3	1	13.57
	CUST148175	558	Low	5	1	13.58
	CUST130793	546	Low	3	1	13.64

Power BI Dashboard

